

✓ Atividades semanais.

1.) Seleção de linguagem e bibliotecas BLAS.

Nesta etapa deverão ser selecionadas linguagens e bibliotecas candidatas para o desenvolvimento. Deverá ser verificado se as escolhas suportam o desenvolvimento dos requisitos funcionais e não funcionais necessários.

2.) Teste das operações básicas.

Utilizar as bibliotecas selecionadas e realizar as operações básicas a seguir:

$$\begin{aligned}MN &= M * N \\ aM &= a * M \\ Ma &= M * a\end{aligned}$$

Estas operações são todas matriciais, envolvendo matrizes e vetores e devem seguir as regras da Álgebra Linear.

Dados para teste:

[Dados.zip](#)

3.) Implementação do algoritmo CGNR.

Codificar um protótipo com o Algoritmo CGNR. Validar os resultados com os dados experimentais. Medir o tempo total de execução e o consumo de recursos como memória e ocupação de CPU.



4.) Testes de saturação e implementação de rotinas de controle.

Realizar testes de saturação do sistema.

Medir os recursos e projetar um algoritmo para controlar e evitar a saturação.

✓ Avaliações.

A avaliação final será proporcional ao número de funcionalidades apresentadas.

Funcionalidades com problemas de execução e/ou incompletas não serão consideradas na avaliação.

O cálculo da nota para cada atividade seguirá os critérios abaixo:

- R - Atendimento dos requisitos (40%)
- P - Adesão aos padrões (40%)
- I - Apresentação final do relatório comparativo (20%)

Será atribuída uma nota por participação e colaboração em aula e atividades (C).

Nota final da atividade:

$$N_1 = \left(\frac{R}{10} \times 0.4 + \frac{P}{10} \times 0.4 + \frac{I}{10} \times 0.2 \right)$$

$$N_2 = \left(\frac{R}{10} \times 0.4 + \frac{P}{10} \times 0.4 + \frac{I}{10} \times 0.2 \right)$$

Média final:

$$M = \frac{(N_1 + N_2)}{2} \times 0.9 + \bar{C} \times 0.1$$

?

✓ Projeto APS semestral.

Algoritmos e definições.

Siglas

g - Vetor de sinal
H - Matriz de modelo
f - Imagem
S - Número de amostras do sinal
N - Número de elementos sensores

Cálculo do fator de redução (C)

$$C = ||\mathbf{H}^T * \mathbf{H}||_2$$

Cálculo do coeficiente de regularização (λ)

$$\lambda = \max(\text{abs}(\mathbf{H}^T * \mathbf{g})) * 0.10$$

Cálculo do erro (ϵ)

$$\epsilon = ||\mathbf{r}_{i+1}||_2 - ||\mathbf{r}_i||_2$$

Cálculo do ganho de sinal (γ)

```
for c = 1 .. N
  for l = 1 .. S
     $\gamma_l = 100 + 1/20 * l * \sqrt{l}$ 

     $\mathbf{g}_{l,c} = \mathbf{g}_{l,c} * \gamma_l$ 
```



Algoritmo 1: CGNE (Conjugate Gradient Method Normal Error)

```
 $\mathbf{f}_0 = 0$ 

 $\mathbf{r}_0 = \mathbf{g} - \mathbf{H}\mathbf{f}_0$ 

 $\mathbf{p}_0 = \mathbf{H}^T \mathbf{r}_0$ 

for i = 0, 1, ..., until convergence

   $\alpha_i = \frac{\mathbf{r}_i^T \mathbf{r}_i}{\mathbf{p}_i^T \mathbf{p}_i}$ 

   $\mathbf{f}_{i+1} = \mathbf{f}_i + \alpha_i \mathbf{p}_i$ 

   $\mathbf{r}_{i+1} = \mathbf{r}_i - \alpha_i \mathbf{H} \mathbf{p}_i$ 

   $\beta_i = \frac{\mathbf{r}_{i+1}^T \mathbf{r}_{i+1}}{\mathbf{r}_i^T \mathbf{r}_i}$ 

   $\mathbf{p}_{i+1} = \mathbf{H}^T \mathbf{r}_{i+1} + \beta_i \mathbf{p}_i$ 
```

Algoritmo 1: CGNR (Conjugate Gradient Normal Residual) (Saad2003, p. 266)

```

$$\mathbf{f}_0 = \mathbf{0}$$

$$\mathbf{r}_0 = \mathbf{g} - \mathbf{H}\mathbf{f}_0$$

$$\mathbf{z}_0 = \mathbf{H}^T \mathbf{r}_0$$

$$\mathbf{p}_0 = \mathbf{z}_0$$
for  $i = 0, 1, \dots$ , until convergence
$$\mathbf{w}_i = \mathbf{H}\mathbf{p}_i$$

$$\alpha_i = \|\mathbf{z}_i\|_2^2 / \|\mathbf{w}_i\|_2^2$$

$$\mathbf{f}_{i+1} = \mathbf{f}_i + \alpha_i \mathbf{p}_i$$

$$\mathbf{r}_{i+1} = \mathbf{r}_i - \alpha_i \mathbf{w}_i$$

$$\mathbf{z}_{i+1} = \mathbf{H}^T \mathbf{r}_{i+1}$$

$$\beta_i = \|\mathbf{z}_{i+1}\|_2^2 / \|\mathbf{z}_i\|_2^2$$

$$\mathbf{p}_{i+1} = \mathbf{z}_{i+1} + \beta_i \mathbf{p}_i$$

```

Requisitos não funcionais e restrições.

Cada imagem deverá conter no mínimo os seguintes dados:

- Identificação do algoritmo utilizado
- Data e hora do início da reconstrução;
- Data e hora do término da reconstrução;
- Tamanho em pixels;
- O número de iterações executadas.

Cliente.



Implementar uma aplicação cliente com as seguintes características:

1. Enviar uma sequência de sinais ($\mathbf{g}$) em intervalos de tempo aleatórios;
2. O ganho de sinal e o modelo da imagem deverão ser definidos aleatoriamente;
3. Gerar um relatório com todas as imagens reconstruídas com as seguintes informações: imagem gerada, número de iterações e tempo de reconstrução;
4. A sequência de sinais ($\mathbf{g}$) enviados deverão ser os mesmos para as duas versões de algoritmos de reconstrução.

Servidor.

Implementar um servidor para reconstrução de imagens:

1. Implementar uma versão utilizando uma linguagem interpretada e não fortemente tipada.
2. Implementar uma versão utilizando uma linguagem compilada e fortemente tipada.
3. Executar o algoritmo de reconstrução;
4. Executar até que o erro (ϵ) seja menor do que $1e10^{-4}$ ou o número de iteração chegar a 10.
5. Criar um relatório comparativo analisando os resultados obtidos com as duas versões.
6. O objetivo principal deve ser reconstruir o maior número de imagens no menor tempo possível.

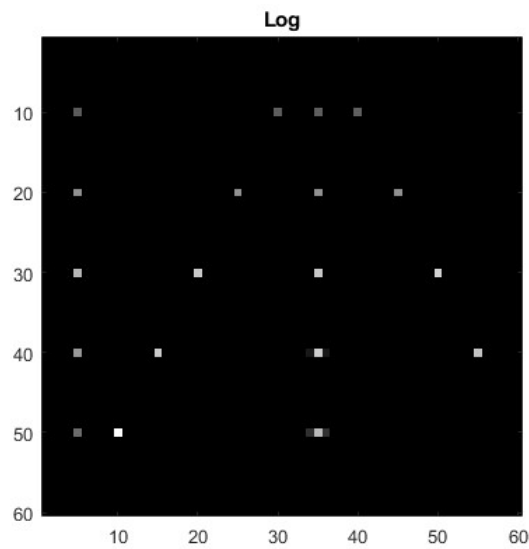
✓ Dados de testes para o modelo 1.

Matriz modelo para imagens de 60 x 60 pixels.

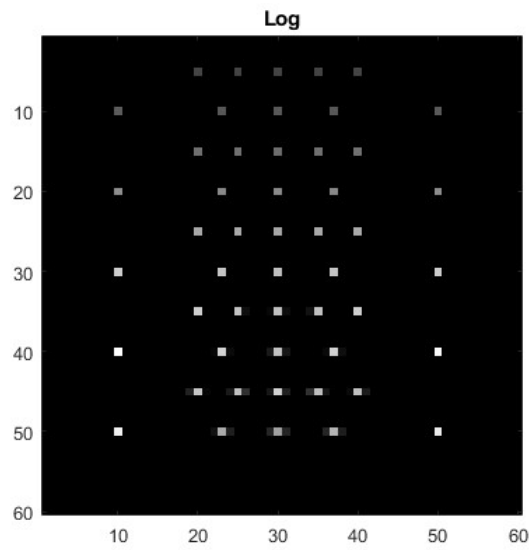
[Matriz H \(Modelo: 50816 x 3600, 60 x 60 pixels, S = 794, N = 64\)](#)

Dados para as imagens.

1.) [Sinal para imagem 1 \(60 x 60 pixels\).](#)



2.) [Sinal para a imagem 2 \(60 x 60 pixels\).](#)



3.) [Sinal para imagem 3 \(60 x 60 pixels\).](#)



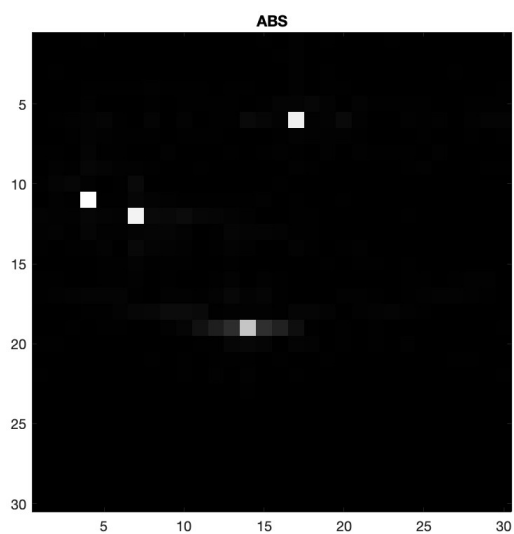
▼ **Dados de testes para o modelo 2.**

Matriz modelo para imagens de 30 x 30 pixels.

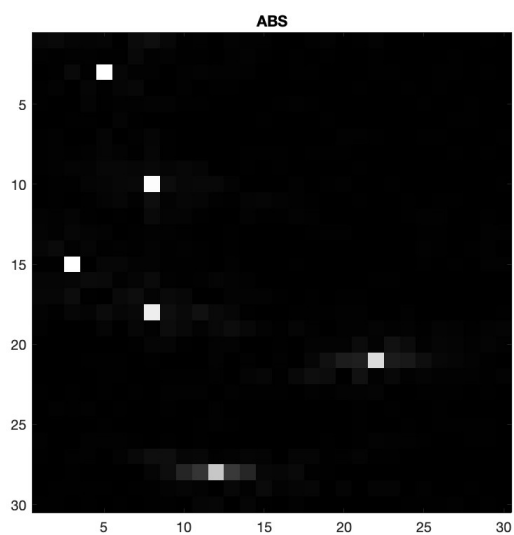
[Matriz H \(Modelo: 27904 x 900, 30 x 30 pixels, S = 436, N = 64\).](#)

Dados para as imagens.

[1.\) Sinal para a imagem 1 \(30 x 30 pixels\).](#)



[2.\) Sinal para imagem 2 \(30 x 30 pixels\).](#)



[3.\) Sinal para imagem 3 \(30 x 30 pixels\).](#)



▼ Calendário de entregas.